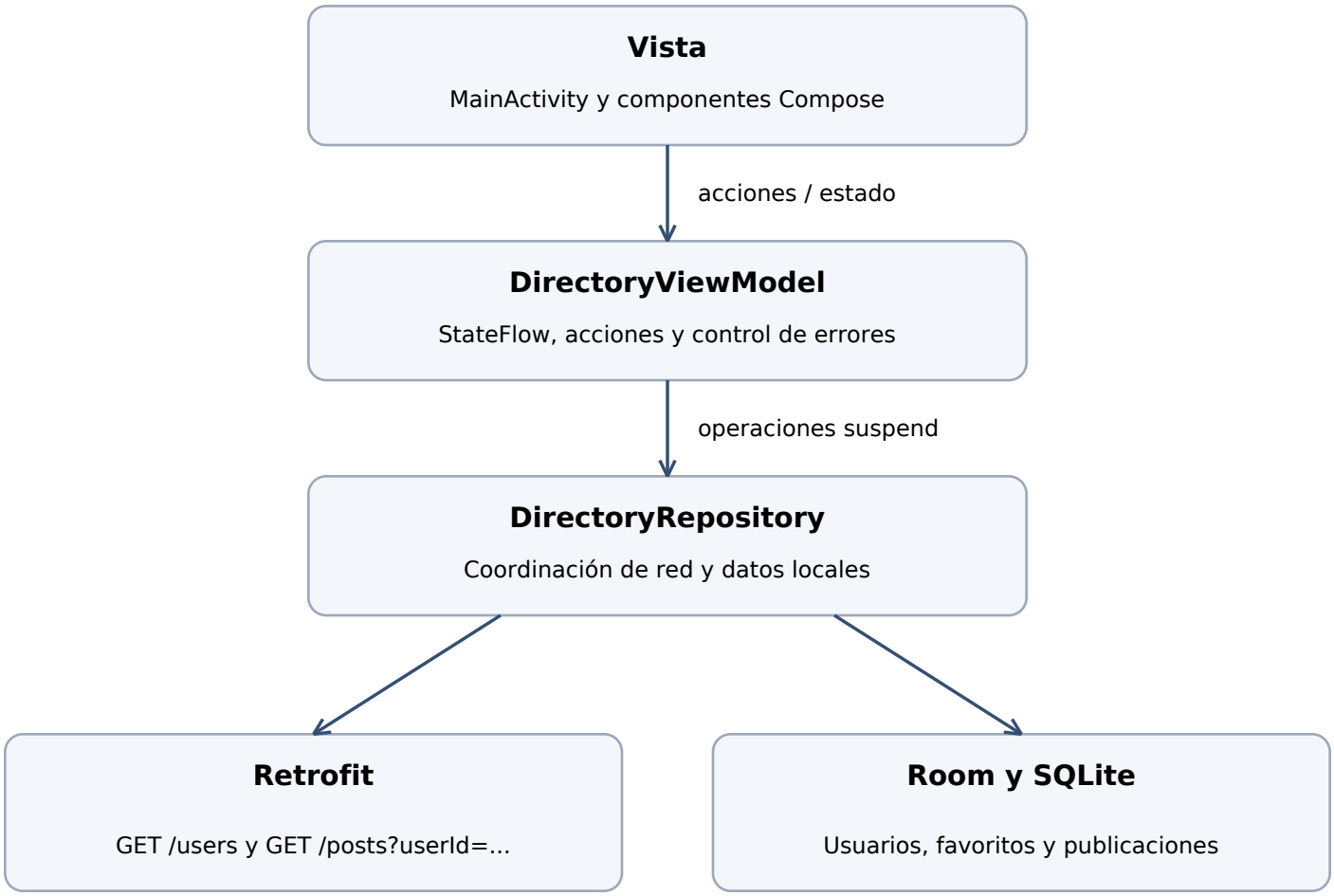
Directorio de usuarios con JSONPlaceholder

Informe de implementación | Kotlin, Retrofit, Room y Jetpack Compose

La aplicación consulta usuarios, muestra sus datos y publicaciones y permite guardar favoritos en el dispositivo. Implementa carga visible, reintento ante errores y consulta local sin Internet. Es un proyecto Android independiente de Mi Recetario, diseñado para abrirse en Android Studio.

Arquitectura MVVM con repositorio



La vista envía acciones al ViewModel y observa su StateFlow. El repositorio consulta la API y escribe en Room. Los Flow de Room actualizan el ViewModel y la interfaz. Connectivity observa el acceso a Internet mediante ConnectivityManager.

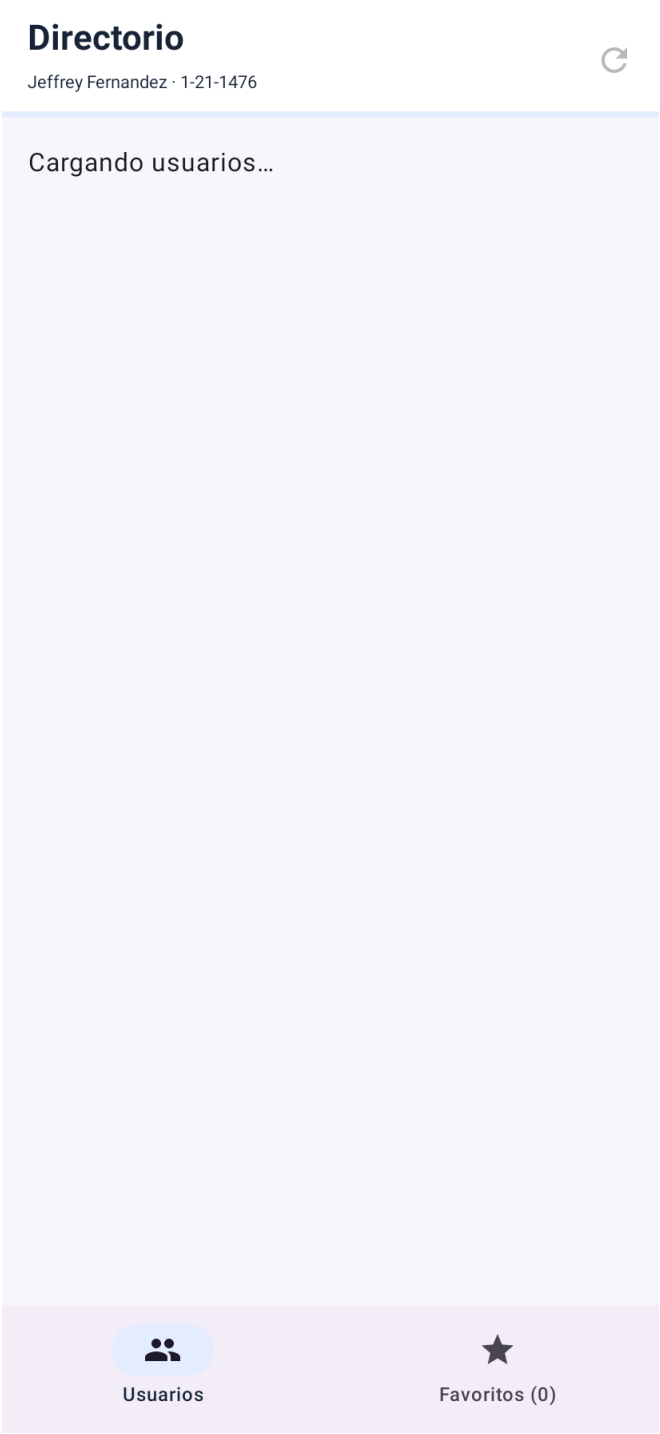
Se conserva el registro completo de cada usuario, no solo el identificador favorito. Las publicaciones quedan disponibles offline después de abrir el detalle y descargarlas. Al actualizar el directorio no se elimina la tabla de favoritos.

Datos: JSONPlaceholder es una API pública de demostración. No se solicitan credenciales y todos sus usuarios y publicaciones son ficticios. [1]

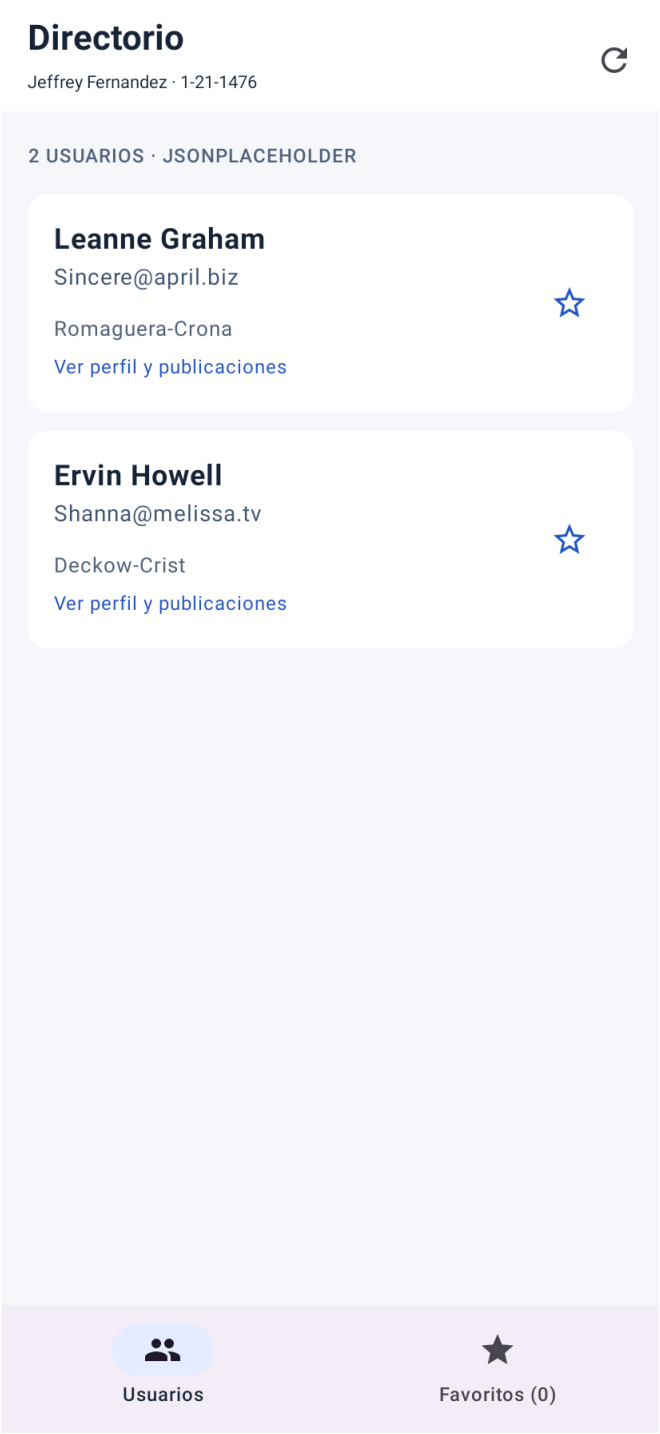
Evidencias de carga y éxito

Capturas del componente Compose real renderizado en pruebas Android con Robolectric y estados controlados. No son capturas de un teléfono físico ni demuestran una llamada en vivo desde el dispositivo. La API real se verificó por separado.

1 Cargando usuarios



2 Usuarios disponibles

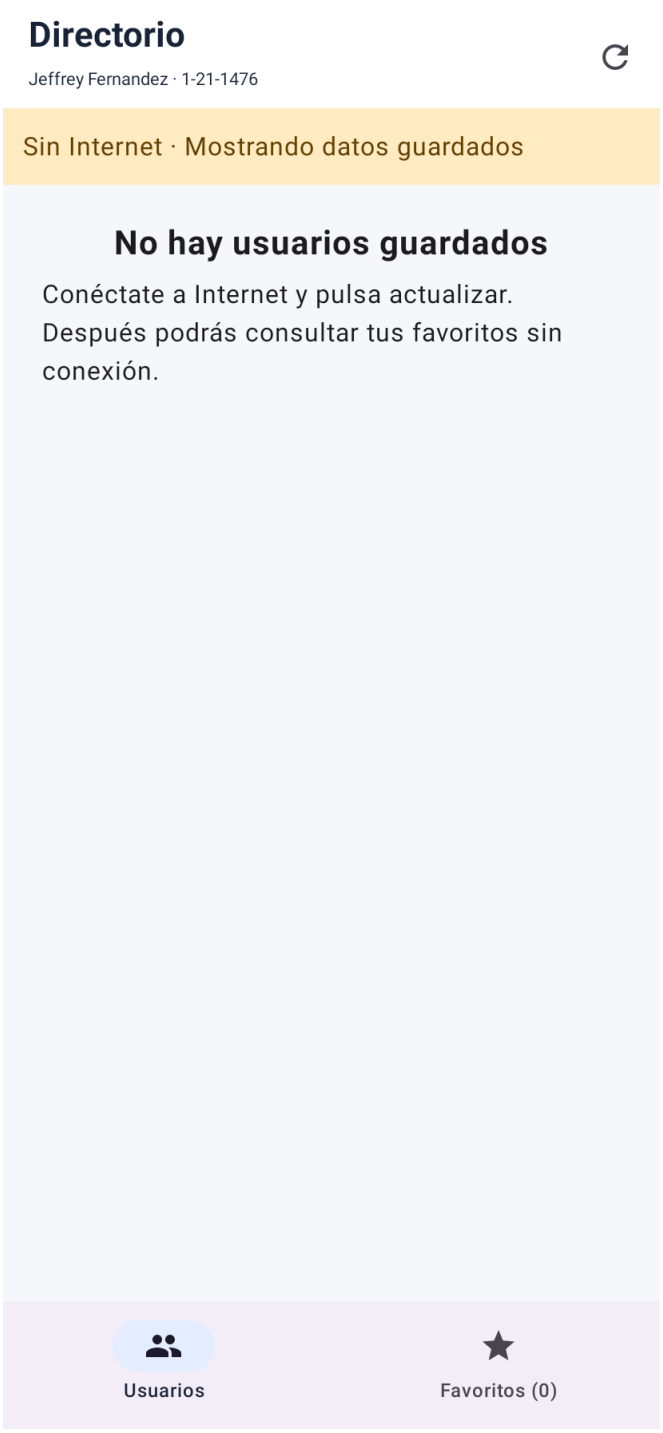


La carga muestra un indicador de progreso y un mensaje. En éxito, cada tarjeta contiene nombre, correo y empresa. Al pulsarla se abre el detalle; la estrella permite guardar el usuario.

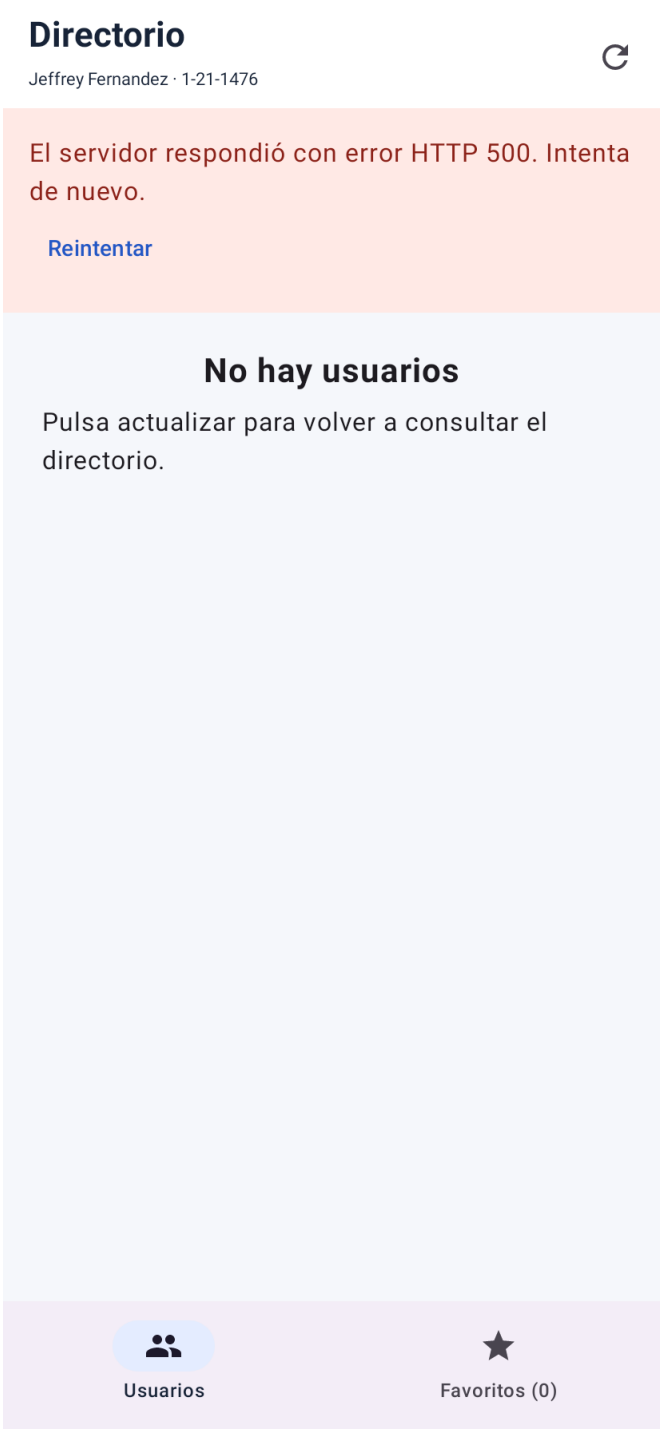
Evidencias de conectividad y errores

Los estados de fallo se fuerzan en la prueba de interfaz para comprobar el texto y la posibilidad de recuperarse. La app conserva los datos locales disponibles y evita mostrar una carga infinita.

3 Sin conexión y sin caché



4 Error del servidor



Si no hay Internet se muestra un aviso explícito. Si existe una copia local, continúa visible; si no existe, se explica cómo obtenerla. El error HTTP ofrece Reintentar. Las excepciones se capturan y CancellationException se propaga para respetar la cancelación de corrutinas.

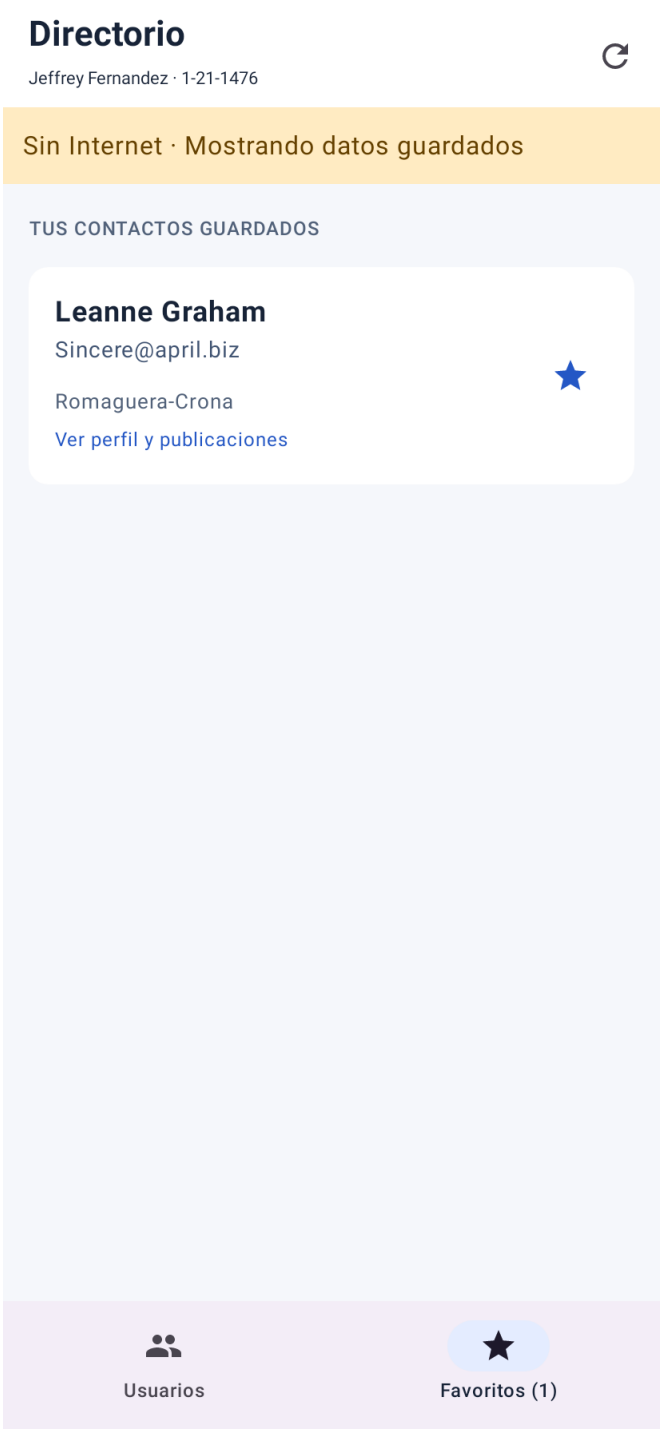
Evidencias de detalle y favoritos

La prueba marca un usuario como favorito y cambia a la pestaña correspondiente. Después abre el detalle. El cuerpo de la publicación usado en esta evidencia es un extracto de un ejemplo de JSONPlaceholder.

5 Datos y publicaciones



6 Favoritos sin Internet



El perfil incluye correo, teléfono, web, empresa, dirección y coordenadas. Los favoritos se consultan filtrando los usuarios locales por los IDs guardados. Las publicaciones no descargadas muestran un aviso cuando no hay Internet.

Código de consumo de la API

Retrofit declara dos consultas GET. @Query construye el parámetro userId y Gson convierte las respuestas JSON a clases Kotlin. Las funciones suspend se invocan desde viewModelScope sin bloquear la interfaz. [1, 2]

```
package edu.utesa.directorio.data

import okhttp3.OkHttpClient
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import retrofit2.http.GET
import retrofit2.http.Query
import java.util.concurrent.TimeUnit

interface DirectoryApi {
    @GET("users")
    suspend fun users(): List<User>

    @GET("posts")
    suspend fun posts(@Query("userId") userId: Int): List<Post>

    companion object {
        fun create(baseUrl: String = "https://jsonplaceholder.typicode.com/"): DirectoryApi {
            val client = OkHttpClient.Builder()
                .connectTimeout(15, TimeUnit.SECONDS)
                .readTimeout(20, TimeUnit.SECONDS)
                .callTimeout(30, TimeUnit.SECONDS)
                .build()
            return Retrofit.Builder().baseUrl(baseUrl).client(client)
                .addConverterFactory(GsonConverterFactory.create())
                .build().create(DirectoryApi::class.java)
        }
    }
}
```

Permisos y comunicación

AndroidManifest declara android.permission.INTERNET y android.permission.ACCESS_NETWORK_STATE. Son permisos normales y no necesitan un diálogo de autorización en tiempo de ejecución. La API se consulta por HTTPS y se desactiva el tráfico HTTP sin cifrar.

Se fijan tiempos máximos de conexión, lectura y petición. Los fallos HTTP, de red y de procesamiento se convierten en mensajes de interfaz con posibilidad de reintento.

Modelo y datos visibles

User contiene id, name, username, email, phone, website, address y company. Company conserva name, catchPhrase y bs. Post contiene userId, id, title y body. El filtro de publicaciones se aplica en el servidor mediante /posts?userId={id}.

Código de persistencia local con Room

Se reutiliza la estrategia de almacenamiento local de la actividad anterior: Room sobre SQLite. Las tablas separan datos de usuarios, favoritos y publicaciones guardadas. La transacción alterna el favorito de forma consistente. [3]

```
package edu.utesa.directorio.data

import androidx.room.*
import kotlinx.coroutines.flow.Flow

@Entity(tableName = "users")
data class UserRow(@PrimaryKey val id: Int, val json: String)
@Entity(tableName = "favorites")
data class FavoriteRow(@PrimaryKey val userId: Int)
@Entity(tableName = "post_cache")
data class PostCacheRow(@PrimaryKey val userId: Int, val json: String)

@Dao
interface DirectoryDao {
    @Query("SELECT * FROM users ORDER BY id")
    fun observeUsers(): Flow<List<UserRow>>
    @Query("SELECT userId FROM favorites")
    fun observeFavorites(): Flow<List<Int>>
    @Query("SELECT * FROM post_cache WHERE userId = :id")
    suspend fun cachedPosts(id: Int): PostCacheRow?
    @Upsert suspend fun saveUsers(users: List<UserRow>)
    @Upsert suspend fun savePosts(posts: PostCacheRow)
    @Insert(onConflict = OnConflictStrategy.IGNORE)
    suspend fun addFavorite(favorite: FavoriteRow)
    @Query("DELETE FROM favorites WHERE userId = :id")
    suspend fun removeFavorite(id: Int)
    @Query("SELECT EXISTS(SELECT 1 FROM favorites WHERE userId = :id)")
    suspend fun isFavorite(id: Int): Boolean

    @Transaction
    suspend fun toggleFavorite(id: Int) {
        if (isFavorite(id)) removeFavorite(id) else addFavorite(FavoriteRow(id))
    }
}

@Database(entities = [UserRow::class, FavoriteRow::class, PostCacheRow::class],
    version = 1, exportSchema = false)
abstract class DirectoryDatabase : RoomDatabase() {
    abstract fun dao(): DirectoryDao
}
```

Repositorio y estrategia sin conexión

```
package edu.utesa.directorio.data

import com.google.gson.Gson
import com.google.gson.reflect.TypeToken
import kotlinx.coroutines.flow.map

class DirectoryRepository(private val api: DirectoryApi, private val dao: DirectoryDao) {
    private val gson = Gson()
    val users = dao.observeUsers().map { rows ->
        rows.map { gson.fromJson(it.json, User::class.java) }
    }
    val favorites = dao.observeFavorites().map { it.toSet() }

    suspend fun refreshUsers() {
        val users = api.users()
        require(users.all { it.id > 0 && !it.name.isNullOrBlank() &&
            !it.email.isNullOrBlank() && it.company != null }) { "Datos inválidos" }
        // No se reemplaza la tabla de favoritos al actualizar el directorio.
        dao.saveUsers(users.map { UserRow(it.id, gson.toJson(it)) })
    }
    suspend fun toggleFavorite(id: Int) = dao.toggleFavorite(id)
    suspend fun cachedPosts(id: Int): List<Post>? = dao.cachedPosts(id)?.let {
        gson.fromJson(it.json, object : TypeToken<List<Post>>() {}.type)
    }
    suspend fun refreshPosts(id: Int): List<Post> {
        val posts = api.posts(id)
        require(posts.all { it.userId == id && !it.title.isNullOrBlank() && it.body != null })
        dao.savePosts(PostCacheRow(id, gson.toJson(posts)))
        return posts
    }
}
```

Decisiones de almacenamiento

Los datos de usuario y las publicaciones se serializan como JSON dentro de registros Room; el identificador es la clave primaria. Esto simplifica la práctica y mantiene los campos anidados. Para búsquedas complejas o catálogos grandes convendría normalizar el esquema.

La interfaz observa Room. Al iniciar se intenta actualizar y también se permite hacerlo manualmente. Si se recupera la conexión, se reintenta la carga. Los favoritos se conservan al reiniciar; las publicaciones solo se guardan después de consultarlas.

Al no reemplazar toda la caché, un usuario retirado del servidor podría seguir visible localmente. Para esta API de demostración estática se acepta esa limitación; una app de producción necesitaría caducidad y reglas de eliminación.

Ciclo de vida y validación

Relación entre ciclo de vida y carga de datos

onCreate: MainActivity crea la interfaz y obtiene DirectoryViewModel. El init del ViewModel inicia la observación local y solicita la carga inicial. La red no se ejecuta directamente en cada recomposición.

onStart y onStop: collectAsStateWithLifecycle observa el estado mientras la actividad está al menos STARTED y suspende esa recolección al salir de ese estado. La carga en viewModelScope puede seguir hasta completarse; no se repite automáticamente por regresar a primer plano. [4]

onResume y onPause: la pantalla gana o pierde la interacción. No se usan estos eventos para recargar de forma indiscriminada. Se actualiza por inicio, acción manual o recuperación de conexión.

Rotación: se recrea la Activity, pero se conserva el ViewModel. La solicitud en curso y los datos no se reinician por la rotación. SavedStateHandle conserva el ID seleccionado y la pestaña ante una recreación compatible del proceso. Room conserva los datos duraderos.

onDestroy: puede ocurrir por rotación o salida definitiva. Cuando el ViewModel se elimina, viewModelScope cancela sus tareas; callbackFlow libera el observador de conectividad. No se depende de onDestroy para guardar favoritos: se escriben en el momento de pulsar la estrella.

Verificaciones

Se incluyen pruebas de Retrofit con MockWebServer, de persistencia al cerrar y reabrir Room y de los estados visuales. Los registros de ejecución acompañan al proyecto en docs/verificacion. La respuesta real de JSONPlaceholder se contrastó: 10 usuarios y 10 publicaciones del usuario 1.

Las capturas proceden de pruebas con estado controlado. Antes de entregar, conviene ejecutar la lista manual del README en un Android o emulador: carga en vivo, rotación, cierre, reinicio, favoritos offline y recuperación de conexión.

Conclusiones

Separar vista, estado y datos permite mostrar resultados locales mientras se consulta la red y controlar los errores sin cerrar la app. Room hace que los favoritos duren más que la Activity; ViewModel conserva el estado durante cambios de configuración. Una interfaz offline debe distinguir datos guardados de información que aún no se descargó.

Repositorio y puesta en marcha

Enlace al repositorio

Pendiente de publicación. No se ha creado ni verificado un repositorio remoto. El proyecto se entrega en ZIP, con README, Gradle Wrapper, pruebas y un flujo de GitHub Actions para compilar. No se incluye una URL inventada.

Nombre sugerido: **directorio-jsonplaceholder**. Tras crearlo en tu cuenta de GitHub, copia su enlace real y añádelo aquí antes de entregar al profesor. Este es el único dato externo que no puede completarse sin publicar en la cuenta.

Abrir el proyecto

1. Extraer DirectorioUsuarios.zip.
2. En Android Studio, elegir Open y abrir la carpeta que contiene settings.gradle.kts.
3. Usar JDK 17 e instalar Android SDK 35 cuando se solicite.
4. Seleccionar un emulador o Android API 26 o superior y pulsar Run.
5. Consultar usuarios con Internet, guardar favoritos y probar el modo sin conexión.

Publicar en GitHub

Crear un repositorio y subir el contenido del proyecto. El README contiene los comandos git init, git add, git commit, git remote y git push. No se deben subir local.properties, la carpeta .gradle ni las carpetas build. El archivo .gitignore ya las excluye.

Fuentes

[1] JSONPlaceholder. API pública y ejemplos. https://jsonplaceholder.typicode.com/

[2] Square. Retrofit. https://github.com/square/retrofit

[3] Android Developers. Room. https://developer.android.com/training/data-storage/room

[4] Android Developers. ViewModel. https://developer.android.com/topic/libraries/architecture/viewmodel

[5] Android Developers. Connect to the network. https://developer.android.com/develop/connectivity/network-ops/connecting

[6] Robolectric. Native graphics. https://robolectric.org/graphics/